Finally, create the routing function that maps to the GET API that was proposed.

```
app.get('/user', (req, res) => {
    const phone = req.query.phone
    controller.searchByPhoneNumber(phone).then(users => {
        res.json(users);
    })
});
```

Once everything is ready, start the *express* object to accept the HTTP requests.

```
app.listen(port, () => console.log(`UMS Search Service
running on port ${port}!`))
```

The code is ready! It can be run standalone on the Node. However, we want it to be deployed as containers on Kubernetes.

Containerisation

The manifest for Docker containerisation is straightforward. Observe that the configuration is also being supplied.

```
FROM node:latest
WORKDIR /app
ENV DB_HOST="mysqldb"
ENV DB_USER="root"
ENV DB_PASSWORD="admin"
ENV DB_DATABASE="glarimy"
ENV PORT=8080
COPY . /app
RUN npm install
CMD node /app/src/index.js
EXPOSE 8080
```

The directory structure looks as follows:

```
ums-search-service
+ src
  +app
    - UserController.js
    - UserRecord.js
    - Users.js
  +data
    - UserRepository.js
  +domain
    - Name.js
    - PhoneNumber.js
    - User.js
    - package.json
    - Dockerfile
```

Now, build the Docker image by running the following command from the root of the project.

```
docker build -it glarimy/ums-search-service .
```

Once the image is ready, push it to the Docker hub.

```
docker push glarimy/ums-search-service
```

Preparing for Kubernetes

Deploying the *SearchService* on Kubernetes is pretty simple as we already tried our hand at it while deploying the *AddService* and *FindService*. Develop a manifest for deployment in a file. The name of the file can be anything. In our case, it's named *kube-deployment.yml*.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ums-search-service
  labels:
    app: ums-search-service
spec:
  replicas: 3
  selector:
    matchLabels:
      app: ums-search-service
  template:
    metadata:
      labels:
        app: ums-search-service
    spec:
      containers:
        - name: ums-search-service
          image: glarimy/ums-search-service
          ports:
            - containerPort: 8080
          env:
            - name: DB_HOST
              value: "mysqldb"
            - name: DB_USER
              value: "root"
            - name: DB_PASSWORD
              value: "admin"
            - name: DB_DATABASE
              value: "glarimy"
            - name: PORT
              value: "8080"
---
apiVersion: v1
kind: Service
metadata:
```